

Computational Thinking

How to break any problem into steps a computer (or a person) can follow — starting with a cake

Week 3 · Sessions 1 & 2

Where we are in the course



BEFORE

Computers & IT infrastructure

What a computer is made of and how systems connect



TODAY

Computational thinking

How to think about a problem so a computer can help



NEXT

Excel 2

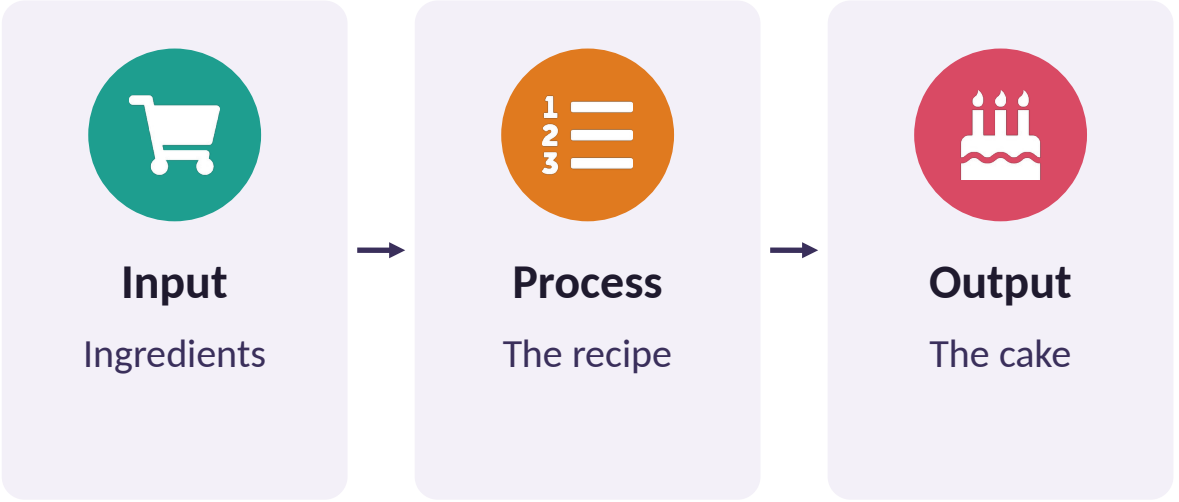
IF, COUNTIF, SUMIF, XLOOKUP and a loan payment with PMT

Last time we looked at the machine. Today we look at the thinking we hand to the machine.

What is computational thinking?

Breaking a problem down and describing the solution so clearly that a computer — or any person — could follow it.

It is not about thinking like a computer. It is about thinking so that a computer can help you.



Remember Input → Process → Output from last week? A recipe is exactly that.

Why should economists care? Budgets, payroll, loans, invoices and forecasts are all step-by-step calculations. If you can describe the steps, Excel (and later PYTHON) can do them for you in seconds.

Daily examples of Computational thinking



Navigation apps

Split the trip into road segments and compare them to find the fastest route.



Music & video suggestions

Spot patterns in what you play and suggest more of the same.



Traffic lights

Green → yellow → red, repeat forever. Sensors add conditions.



Card payments

IF the amount or country looks unusual THEN ask you to confirm.



Ride-hailing prices

IF requests > available drivers THEN the price goes up.



Online shops

IF your basket ≥ 5,000 ALL THEN delivery is free.

Warm-up: Make a cup of tea



In pairs, write instructions for making a cup of tea.
I will follow them exactly, word by word — nothing more, nothing less.

What usually goes wrong

“Put water in the kettle”	Which water? How much? Is the lid open?
“Boil the water”	You never said to switch the kettle on.
“Add sugar”	How much? Before or after the tea bag?

The lesson

A computer does exactly what you say — not what you mean.

The four pillars of computational thinking



Decomposition

Break a big problem into small parts

Cake example

- *Shop*
- *Mix*
- *Bake*
- *decorate*



Pattern recognition

Spot what is similar or repeats

Cake example

Every cake = dry + wet + heat



Abstraction

Keep what matters, ignore the rest

Cake example

Oven temperature matters, bowl colour doesn't



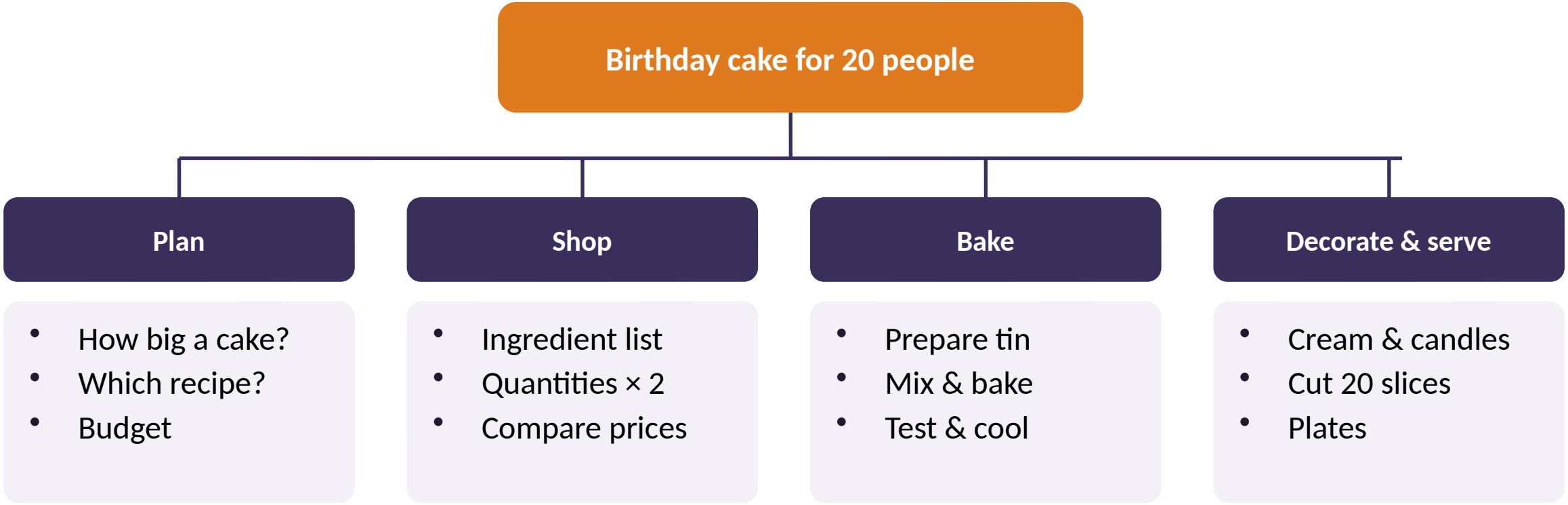
Algorithm

Write clear steps in the right order

Cake example

The recipe itself

Decomposition: break it into smaller problems



Business version: “Open a café” is too big to solve at once. Split it: location, budget, suppliers, staff, marketing — then split again until each piece is easy.

Pattern recognition: spot what repeats

	Chocolate cake	Lemon cake	Muffins	The pattern
Dry	flour, sugar, cocoa	flour, sugar	flour, sugar	dry ingredients
Wet	eggs, milk, butter	eggs, yogurt, oil	eggs, milk, oil	wet ingredients
Flavour	cocoa	lemon zest	blueberries	[flavour]
Heat	180 °C · 35 min	175 °C · 40 min	190 °C · 20 min	[temp] · [time]

One recipe with blanks
Once you see the pattern, you can write one general recipe and just fill in the blanks.

In economics
Every monthly payslip, invoice or loan follows the same pattern — that is exactly why we automate them.

Abstraction: keep only what matters

Keep

- Oven temperature
- Baking time
- Quantities in grams
- Order of the steps

Ignore

- Colour of the bowl
- Brand of the oven
- Music in the kitchen
- Which hand holds the spoon



A metro map is an abstraction
It drops real distances, streets and buildings and keeps only stations and connections — that is why it is easy to read.

Every model is an abstraction: GDP, a budget or an Excel sheet keeps the numbers that matter and ignores the rest.

Algorithm: the recipe is the program

Classic sponge cake

1. Preheat the oven to 180 °C
2. Grease a 24 cm tin
3. Mix 200 g flour, 200 g sugar, 2 tsp baking powder
4. Beat in 4 eggs and 200 g soft butter
5. Pour the batter into the tin
6. Bake for 30 minutes
7. Test with a toothpick; if wet, bake 5 more minutes
8. Cool for 10 minutes, then decorate

A good algorithm is…

1. **Precise**
   “200 g sugar”, not “some sugar”
2. **Ordered**
   You can’t bake before you mix
3. **Finite**
   It must end at some point
4. **Input → output**
   Ingredients in, cake out

Every algorithm uses just three building blocks



Sequence

Do steps one after another

```
Preheat → mix → bake
```



Selection (condition)

Choose what to do based on a yes/no question

```
IF toothpick is wet THEN bake 5 more minutes
```



Repetition (loop)

Repeat steps a number of times or until something is true

```
Beat the eggs UNTIL fluffy
```

From a calculator to Excel to a self-driving car — everything is built from these three.

Conditions: IF … THEN … ELSE

```
IF toothpick comes out clean
   THEN take the cake out
ELSE
   bake 5 more minutes
```

A condition is always a question with only two answers: TRUE or FALSE.

Grades — IF score ≥ 50 THEN “Pass” ELSE “Fail”

Online shop — IF basket ≥ 5,000 ALL THEN free delivery ELSE 300 ALL

Bank — IF monthly payment ≤ 40% of income THEN approve the loan

In Excel (next week):

```
=IF(B2>=50, "Pass", "Fail")
```

Repetition: loops do the boring part



Repeat N times

You know how many times in advance

- Fill 12 cupcake cases
- For each of 30 employees: calculate the salary



Repeat UNTIL

Stop when a condition becomes true

- Beat the eggs until fluffy
- Ask for the PIN until it is correct (max 3 tries)



The infinite loop

“Stir the batter until the cake is ready.”

Stirring never bakes it — so you stir forever. Every loop needs a condition that will eventually be true.

Activity: spot the building blocks

Vending machine algorithm

1	Show the price of the drink	S / C / R ?
2	Wait for a coin	S / C / R ?
3	Add the coin to the total	S / C / R ?
4	Repeat steps 2–3 until total ≥ price	S / C / R ?
5	If total > price, give change	S / C / R ?
6	Drop the drink	S / C / R ?



Your task · 5 min

1. Label each line: Sequence, Condition or Repetition
2. What if the machine is out of drinks? Add the missing step
3. Where could an infinite loop happen?

Session 1 wrap-up

What we learned

4 pillars — Decomposition · Patterns · Abstraction · Algorithms

3 blocks — Sequence · Selection · Repetition

1 rule — Computers do exactly what you say



Exit ticket

Write the algorithm for withdrawing cash from an ATM in at most 8 steps.

Mark at least one condition and one loop.

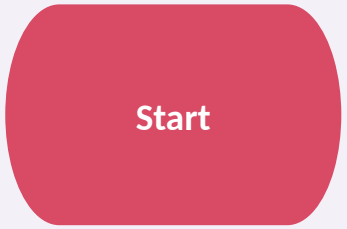
Keep it — we turn it into a flowchart next session.



Flowcharts: drawing the algorithm

Session 2 · from steps on paper to steps in a spreadsheet

Five symbols are all you need



Start / End

Where the algorithm begins and finishes



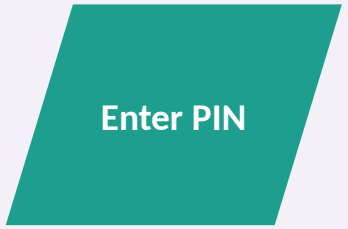
Process

An action or a calculation



Decision

A yes/no question — two arrows out



Input / Output

Data goes in or comes out

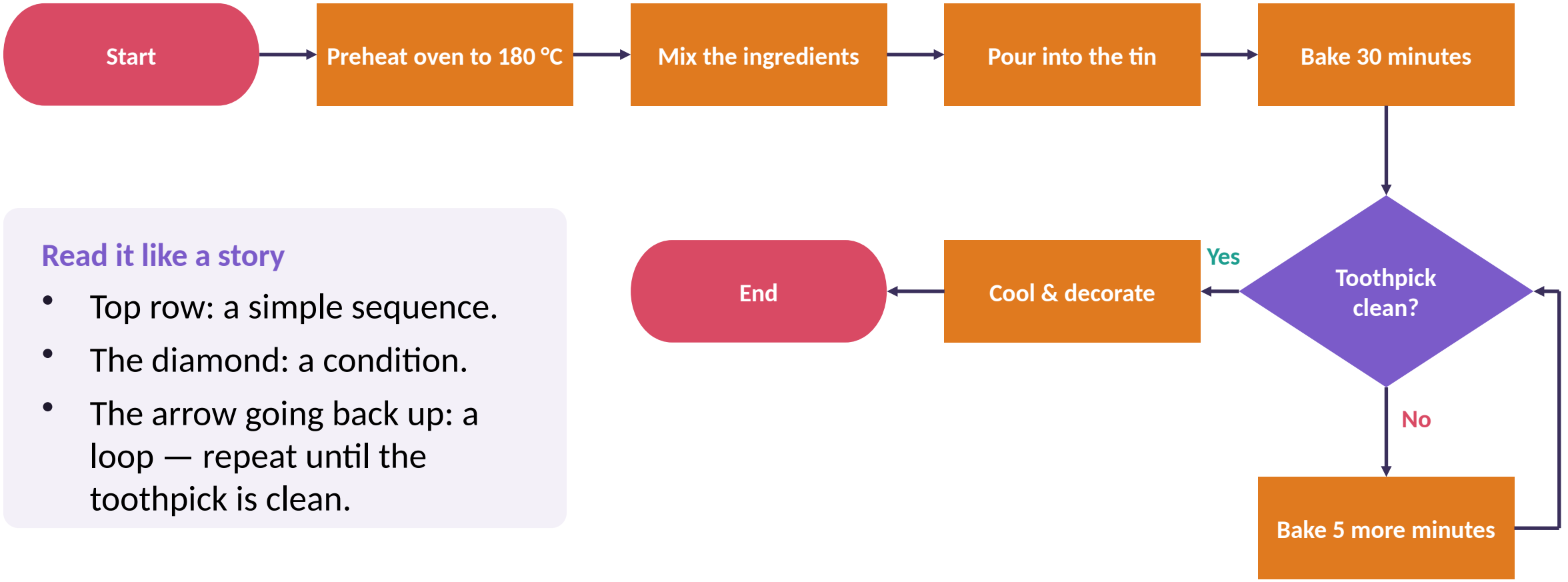


Arrow

Shows the order of the steps

Rule of thumb: every decision diamond has exactly two exits, labelled Yes and No.

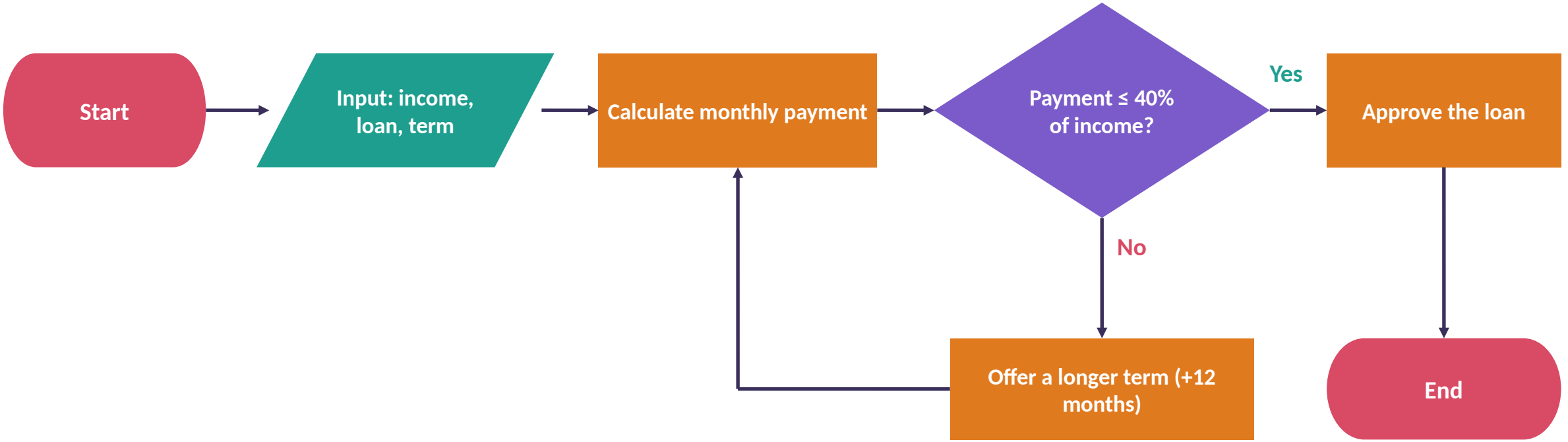
Flowchart: baking the cake



Read it like a story

- Top row: a simple sequence.
- The diamond: a condition.
- The arrow going back up: a loop — repeat until the toothpick is clean.

Flowchart: approving a bank loan



The calculation → **PMT**

The diamond → **IF**

The arrow back → **a loop: try again**

Activity: draw your ATM flowchart



Turn your exit ticket into a flowchart

Pairs · 15 minutes · paper or Insert → Shapes → Flowchart

Checklist

- Starts and ends with a Start/End symbol
- At least one decision, with Yes and No labelled
- A loop: re-enter the PIN, maximum 3 tries
- A check: is the balance ≥ the amount?
- Every arrow points somewhere — no dead ends

Finished early?

Draw a flowchart that gives a student's final grade:

```
≥ 90 → A
≥ 70 → B
≥ 50 → C
otherwise → Fail
```

A spreadsheet is an algorithm you can see

	A	B	C	D
1	**Ingredient**	**Qty**	**Price (ALL)**	**Cost (ALL)**
2	Flour (kg)	0.5	120	60
3	Sugar (kg)	0.3	140	42
4	Eggs (pcs)	4	20	80
5	Butter (kg)	0.25	900	225
6	Cocoa (kg)	0.1	1,200	120
7	**Total**			**527**
8	**Per slice (10)**			**52.7**

1. **Inputs**
   Quantities and prices typed in columns B and C

2. **Process each row** `=B2*C2`
   D2 = B2*C2, then copied down to D6 — the loop!

3. **Combine** `=SUM(D2:D6)`
   Add up the costs

4. **Output** `=D7/10`
   Cost per slice

Change one price → every result updates. The spreadsheet re-runs the whole algorithm for you.

From thinking to Excel functions

IF	**Decide**	Make a yes/no decision	`=IF(B2>=50,"Pass","Fail")`
COUNTIF	**Count with a condition**	How many students passed?	`=COUNTIF(C2:C30,"Pass")`
SUMIF	**Add with a condition**	Total spent on food only	`=SUMIF(A2:A99,"Food",B2:B99)`
XLOOKUP	**Find and fetch**	What is the price of cocoa?	`=XLOOKUP("Cocoa",A2:A6,C2:C6)`
PMT	**Repeated payment**	Monthly payment on a loan	`=PMT(6%/12,36,-1000000)`

Preview: what will my loan cost each month?

Loan amount	1,000,000 ALL
Yearly interest	6%
Duration	3 years

Think in steps

1. Monthly rate = 6% ÷ 12 = 0.5%
2. Number of payments = 3 × 12 = 36
3. Let PMT do the rest

```
=PMT(6%/12, 36, -1000000)
```

30,422 ALL

per month, for 36 months

1,095,190 ALL

paid in total

95,190 ALL

paid in interest

Recap and what comes next

You can now…

- Break a problem into smaller steps
- Write an algorithm with conditions and loops
- Read and draw a flowchart
- See a spreadsheet as a step-by-step calculation

NEXT

Excel worksheet 2

- IF, COUNTIF, SUMIF and XLOOKUP
- A monthly loan payment with PMT

Before each formula, ask: what is the input, what is the condition, what is repeated?